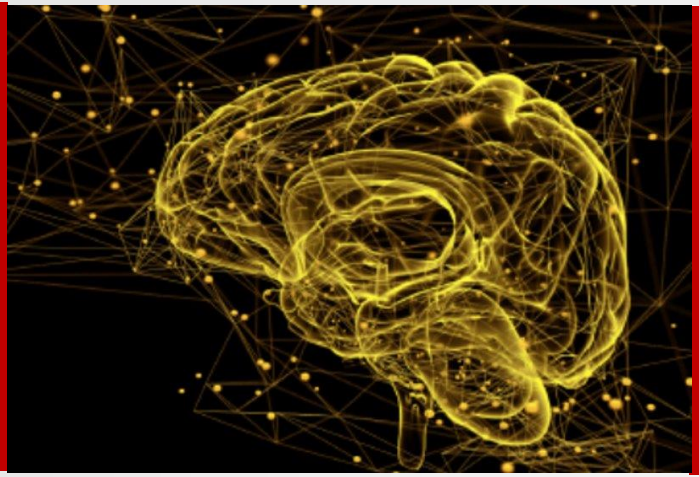
esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

Advanced Learning for text and Graph Data



Chapitre 2: Les Transformers pour Text et Graphe

Unit: IL

United Nations Educational, Scientific and Cultural Organization — UNESCO Chair "Project based learning" ESPRIT School of engineering, Tunisia — uniTwin

EUR-ACE® Délivrée par la Commission des Titres d'Ingénieur

CONFERENCE DES GRANDES ÉCOLES

cdio CONCEIVE DESIGN IMPLEMENT OPERATE

Objectifs

À l'issue de ce chapitre, vous serez en mesure de :

- Identifier les différences entre les architectures neuronales appliquées au **texte** et aux **graphes**.
- Expliquer l'architecture de base des **Transformers** (self-attention, multi-head attention).
- Comparer l'usage des **Transformers** dans le traitement du texte et des graphes.
- Mettre en œuvre des modèles de type **Transformers** pour l'analyse et le traitement de données textuelles (**BERT**).
- Utiliser des Graph Transformer Networks (**GTN**) pour analyser les relations complexes dans les graphes

Plan

1. Les limites des architectures neuronales: RNN et LSTM
2. Expliquer l'architecture de base des Transformers (self-attention, multi-head attention)
3. Mettre en œuvre des modèles de type Transformers pour l'analyse et le traitement de données textuelles (BERT).
4. Comprendre comment les Graph Transformer Networks (GTN) améliorent la représentation des graphes grâce à l'attention

Motivation

Pourquoi ce chapitre ?

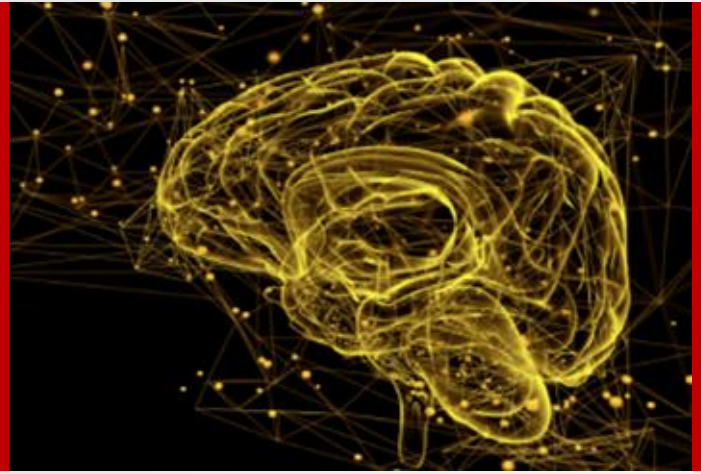
 Les **Transformers** ont révolutionné l'**IA**. Ils s'imposent aujourd'hui comme **architecture universelle** pour unifier le traitement de données :

- Traitement du **texte** (BERT, GPT)
- Analyse de **graphes** complexes (GTN, Graph-BERT)
- Compréhension des **images** (Vision Transformers)

Ce que vous allez découvrir :

- Comment fonctionne le mécanisme **d'attention** qui fait le cœur des Transformers
- Comment l'architecture peut être adaptée aux données textuelles.
- Pourquoi les Transformers surpassent les modèles traditionnels (RNN, CNN, GNN)

esprit
Se former autrement
HONORIS UNITED UNIVERSITIES

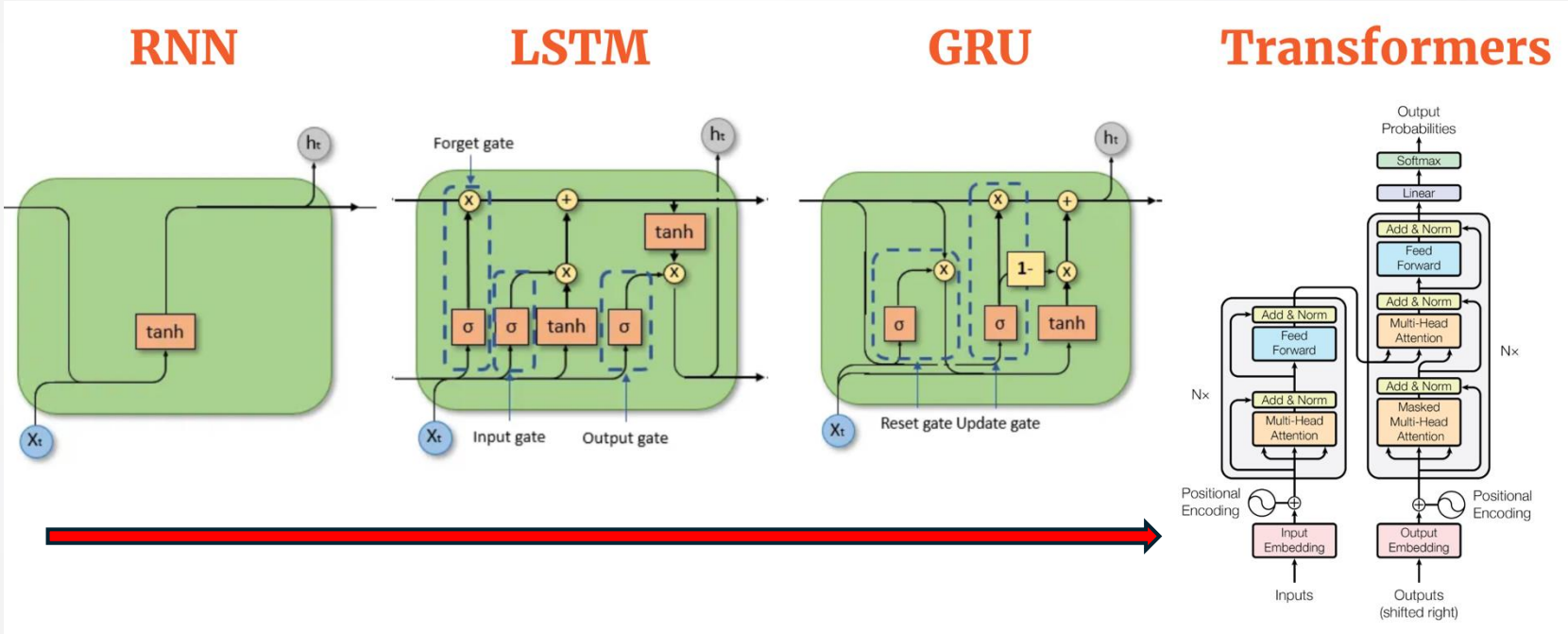


Les limites des architectures neuronales: RNN et LSTM

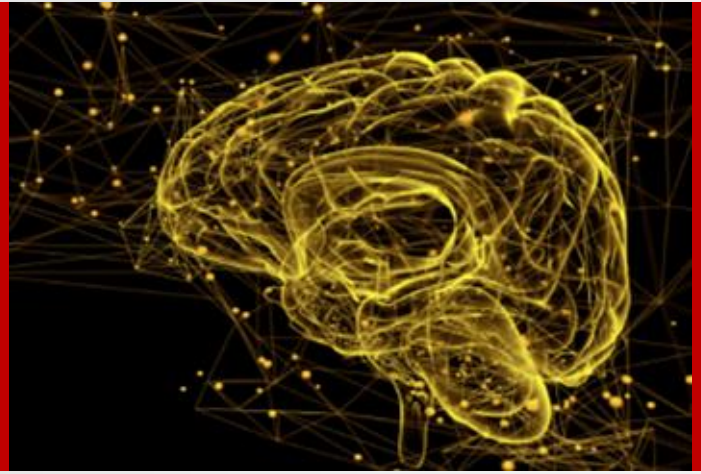
Comparaison : RNN/LSTM vs Transformers

Aspect	RNN / LSTM	Transformers
Parallélisation	Traitement séquentiel (pas de parallélisme naturel) → lent pour de longues séquences.	Traitement parallèle grâce au mécanisme d'attention → entraînement beaucoup plus rapide.
Dépendances longues	Difficulté à capturer des dépendances lointaines (problèmes de vanishing / exploding gradients).	Capturent facilement les dépendances longues grâce à self-attention.
Mémoire contextuelle	Mémoire limitée par l'état caché. Même les LSTM/GRU s'essoufflent sur de très longues séquences.	Chaque token peut accéder directement à tous les autres tokens → meilleure gestion du contexte global.
Complexité computationnelle	Complexité linéaire par rapport à la longueur de la séquence.	Complexité quadratique en fonction de la séquence (mais optimisations récentes comme Linformer réduisent ce coût).
Scalabilité	Difficiles à entraîner sur de grands corpus.	Haute scalabilité → utilisés dans des modèles massifs (BERT, GPT).
Explicabilité	Difficile d'interpréter quels tokens influencent la sortie.	Plus interprétables : on peut visualiser les poids d'attention.

Comparaison : RNN/LSTM vs Transformers



esprit
Se former autrement
HONORIS UNITED UNIVERSITIES



Architecture des Transformers (self-attention, multi-head attention)

Introduction aux transformers

👉 Les Transformers ont été introduits en 2017 par Vaswani et al. dans l'article *"Attention is All You Need"*:*chromeextension://efaidnbmnnnibpcajpcglclefindmkaj/https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf*

👉 Ils ont révolutionné le NLP, puis ont été étendus à la vision, aux graphes et à d'autres domaines.

👉 Contrairement aux RNN et LSTM, les Transformers exploitent **un mécanisme d'attention** qui capture les dépendances globales dans une séquence en parallèle, ce qui permet une **meilleure efficacité et performance**.

- **NLP** : traduction automatique, résumé de texte, classification de documents.
- **Vision** : reconnaissance d'objets, segmentation d'images.
- **BioInformatique** : prédiction de structures protéiques (AlphaFold).
- **Graphes** : classification de molécules, analyse de réseaux sociaux.

Définition

Transformers

- Travaillent sur l'ensemble de la séquence de calcul de l'attention pour toutes les paires de mots, ce qui leur permet d'apprendre les dépendances à long terme.
- Certaines parties de l'architecture peuvent être traitées **en parallèle**, ce qui accélère considérablement la formation sur le site.
- Grâce à leur mécanisme unique d'auto-attention, les modèles de transformateurs offrent une grande capacité de représentation et un grand pouvoir d'expression.
- Les transformateurs sont de **grands modèles d'encodage-décodage** capables de traiter une séquence entière **avec un mécanisme d'attention** sophistiqué.

Architecture du Transformer

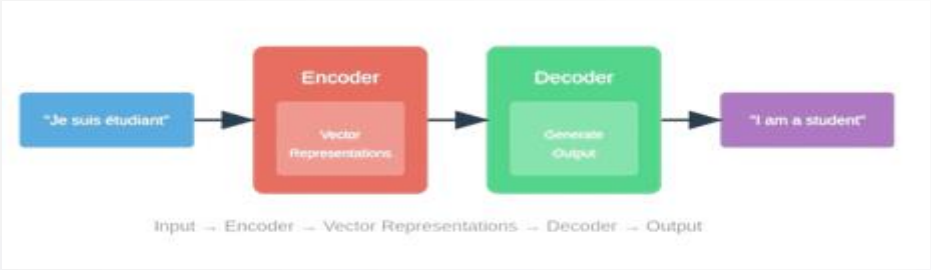
L'architecture Transformer repose sur plusieurs éléments fondamentaux :

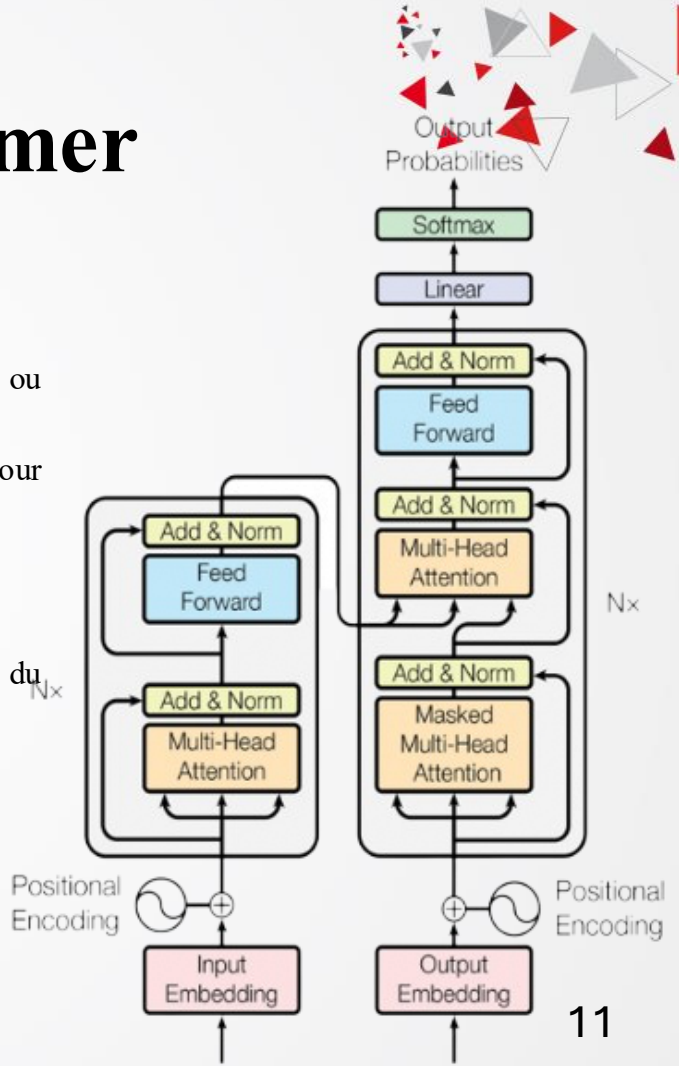
- **Encodage et Décodage** :

➢ Encodeur: Extraction des caractéristiques : L'encodeur transforme le texte ou l'image en une série de valeurs numériques compréhensibles par l'ordinateur.

➢ Décodeur: Génération de la séquence cible : le décodeur utilise ces valeurs pour générer un résultat, comme une traduction ou une prédiction.

- **Encodage de position**
- **Multihead Attention**
- **Couches successives** :Plusieurs couches permettent d'affiner la compréhension du texte à chaque étape.





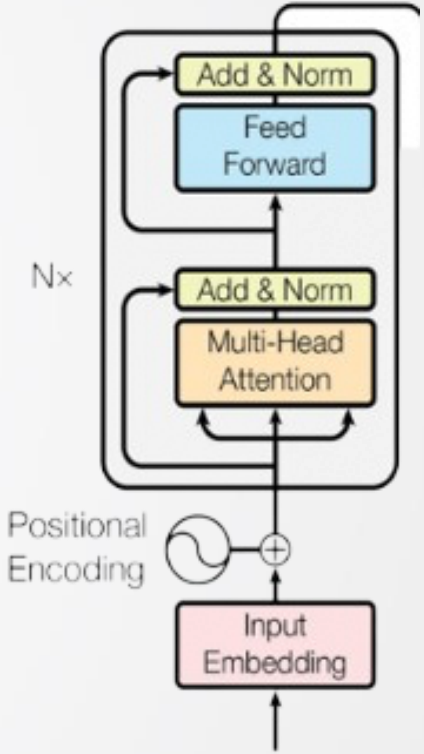
1.Encodeur

→ L'encodeur transforme une suite de tokens en **représentations contextuelles** de même longueur via des couches répétées de **self-attention multi-tête(multi-head attention)** et de **réseaux feed-forward positionnels**, stabilisées par **résidus** + **normalisation.**

1/ Input Embedding (Encodage d'entrée): Transformer les entrées en des vecteurs

2/Positional Encoding : ajoute l'information d'ordre, car le modèle n'est pas séquentiel pour conserver la structure de la phrase.

→ Basé sur les **fonctions sinusoïdales et cosinusales**



1.Encodeur: Le mécanisme d'attention

Le modèle analyse chaque mot d'une phrase et calcule **son importance par rapport aux autres mots pour en comprendre le sens global.**

👉 L'attention calcule une combinaison pondérée des représentations :

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1. **Chaque mot est transformé en trois vecteurs :**

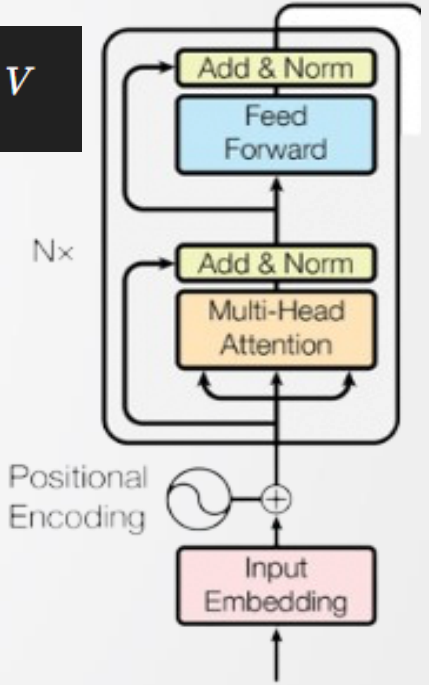
Q : la requête, ce qu'on cherche (cible)
K : la clé, ce qui peut répondre à la requête
V :l'information qu'on transporte
d_k : dimension des clés (normalisation)

2. Dot Product (similarité):

On fait un **produit scalaire (dot product)** entre **Q et K** pour mesurer la similarité.
→ Le modèle mesure combien un mot doit "regarder" un autre.

3. On applique un softmax pour transformer ça en probabilités (poids d'attention).

4. On fait une **somme pondérée des V** en utilisant ces poids.

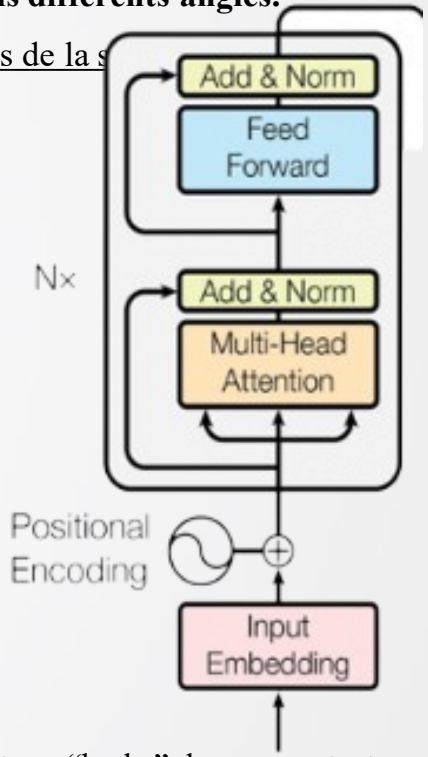


1.Encodeur: Multi-head Attention:

Mécanismes d'attention fonctionnent en parallèle pour **comprendre les relations** entre les **mots sous différents angles.**

→ Chaque token a une nouvelle représentation, enrichie par les informations issues des autres tokens de la s

Exp : « My Apple is broke »
Le modèle veut traduire : *"Mon Apple est cassé"* 💻 ou *"Ma pomme est abîmée"* 🍎. ?
le mecanisme d'attention : la traduction peut être ambiguë.
→Si le modèle ne regarde que le mot "Apple", il **ne comprend pas forcément que c'est un ordinateur**
Multi-head attention



Plusieurs têtes regardent **différents aspects de la phrase** :

Tête	Focalisation	Rôle
1	Mot "Apple"	Correspondance directe
2	Mot suivant "is broke"	Contexte → cassé → ordinateur 💻
3	Mot précédent "My"	Possession → qui possède l'objet
4	Contexte global	Relation sémantique → désambiguation fruit vs PC |

👉 En combinant toutes les têtes : le modèle comprend que **"Apple" = ordinateur** 💻, car un fruit ne serait pas "broke" dans ce contexte

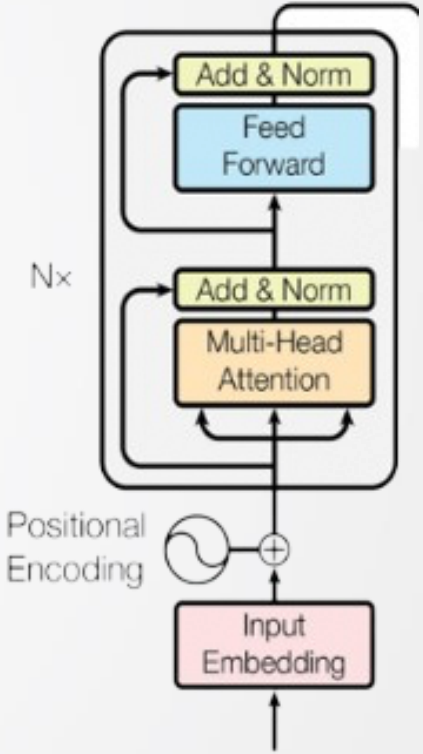
1.Encodeur

4/Add & Norm (Connexion Résiduelle et Normalisation)

5/Feed-Forward Network

- Composé de couche d'entrée, couches cachées et couche de sortie
- Rôle (dans le transformer) :Transformer la représentation de chaque token individuellement

6/Add & Norm (Connexion Résiduelle et Normalisation)



2.Décodeur

Décodeur : Génère la sortie finale (dans le cas de la traduction ou du séquencement) en se basant sur la représentation codée produite par l'encodeur

1/ Outpout (shifted right):la séquence cible (y) décalée vers la droite:Chaque mot (token) est décalé d'une position vers la droite

2/ positional encoding

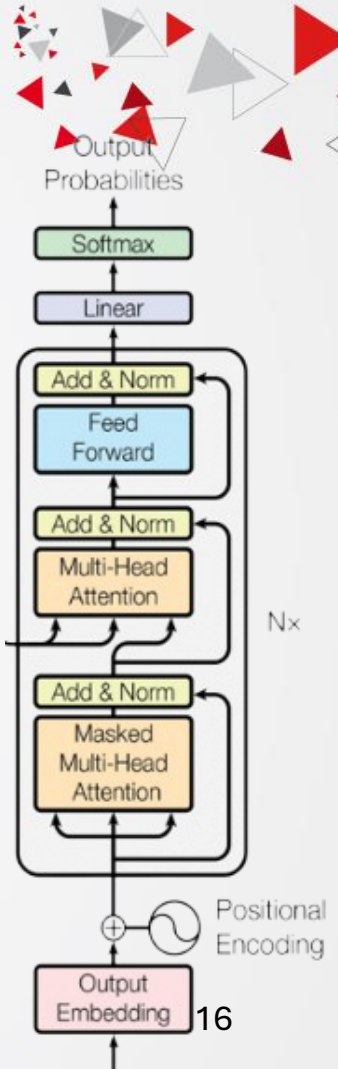
3/ Masked Multi-head Attention: Type de Multi-head attention qui Intègre un masquage pour empêcher l'accès aux futurs tokens dans la séquence cible

4/ Add & Norm

5/ Multi-Head Attention :Cross-Attention (Encodeur-Décodeur Attention)

- Queries (Q): la sortie du self-attention masqué du décodeur
- Keys (K) et Values (V): la sortie finale de l'encodeur

➢ Ajuster la génération de chaque token de sortie en s'appuyant sur la représentation de la phrase source (sortie de l'encodeur)



2.Décodeur

6/ Add & Norm
7/ Feed Forward Network (FNN) : Transformer chaque token indépendamment
8/ Add & Norm
Architecture: Génération de la Sortie

1/ Linear : Projeter les vecteurs de chaque token vers la dimension du vocabulaire
Exemple:
Vocalbulaire: contient 1000 mots→ Chaque token sera projeté sur un vecteur de 1000 dimensions
2/ Softmax : Appliquée sur chaque vecteur projeté

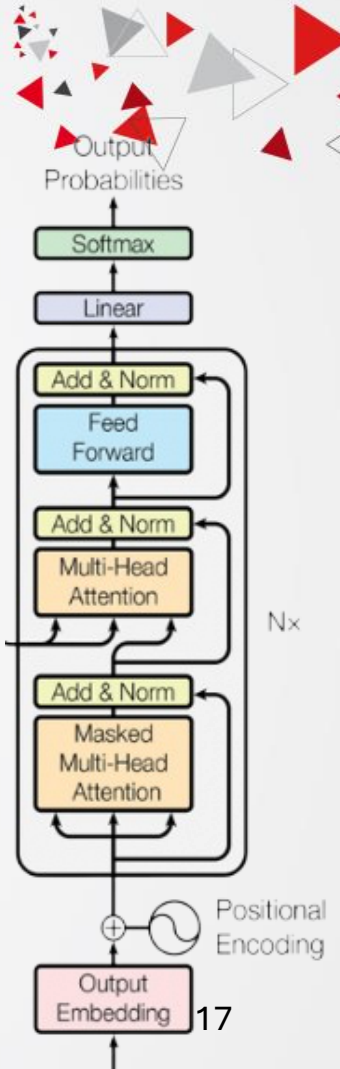
- Le mot avec la probabilité la plus élevée sera sélectionné comme prochain mot de la séquence de sortie

génération de la sortie (texte) mot par mot :

Exemple:

- **Phrase d'entrée :** "Hello how are you?"
- **Phrase de sortie mot par mot:**
  Bonjour
  comment
  ca
  va
  ?

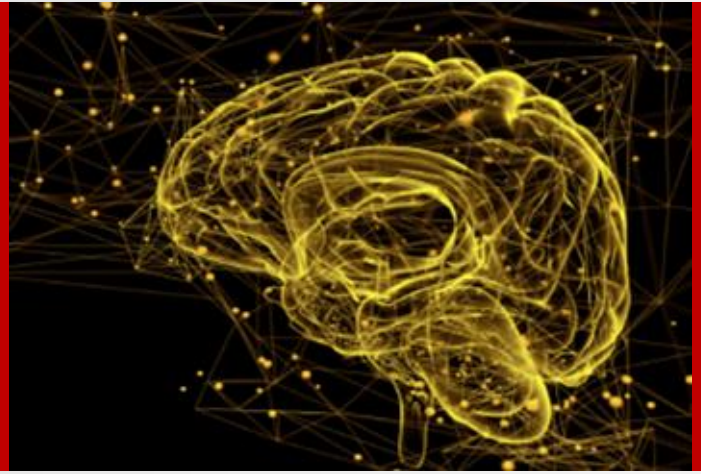
https://colab.research.google.com/drive/1KLuxUNe1IMDyxu6jFsK66JyYIKdf7slo?usp=sharing



Modèles dérivés

Type	Définition	Mode d'utilisation	Exemples
Encoder	Utilise la partie encoder pour apprendre les représentations contextuelles d'un texte (embeddings riches).	Tâches de compréhension du langage (NLP) : classification de texte, analyse de sentiments, NER (Named Entity Recognition), recherche sémantique, QA basé sur passage.	BERT, RoBERTa, ALBERT, DistilBERT
Decoder	Utilise uniquement la partie decoder, avec un mécanisme d'attention causale (regarde uniquement le passé)	Tâches de génération de texte : auto-complétion, dialogue, rédaction de texte, code génération.	GPT (1,2,3,4,5), LLaMA, Mistral
Encoder-Decoder (Seq2Seq)	Combine un encoder (qui comprend l'entrée) et un decoder (qui génère la sortie), avec des couches d'attention entre les deux.	Ttâches de transformation séquence → séquence : traduction, résumé automatique, génération de réponses conditionnées, RAG.	T5, BART, mBART, MarianMT

esprit
Se former autrement
HONORIS UNITED UNIVERSITIES



GTN Graph Transformer Network

Définition de GTN

Graph Transformer Network

- C’est un type de réseau qui combine **les idées des Transformers** et des **Graph Neural Networks (GNN)**.
- Un **Graph Transformer** adapte le mécanisme d’attention des Transformers aux **données en graphe**. Il intègre directement dans ses couches :
  - des **relations structurelles** (nœuds, arêtes),
  - des **dépendances longues**,
  - et gère des **graphes hétérogènes**.
  - Permet un **modèle parallèle et scalable**, bien adapté aux **données relationnelles complexes**.

Définition de GTN

Les Graph Transformer Networks (GTN) ont été introduits afin de :

- **Améliorer l'expressivité:**

  Le mécanisme d'attention permet aux GTN de **pondérer les relations** selon leur pertinence, offrant une **vue plus fine et plus riche** de la structure du graphe.

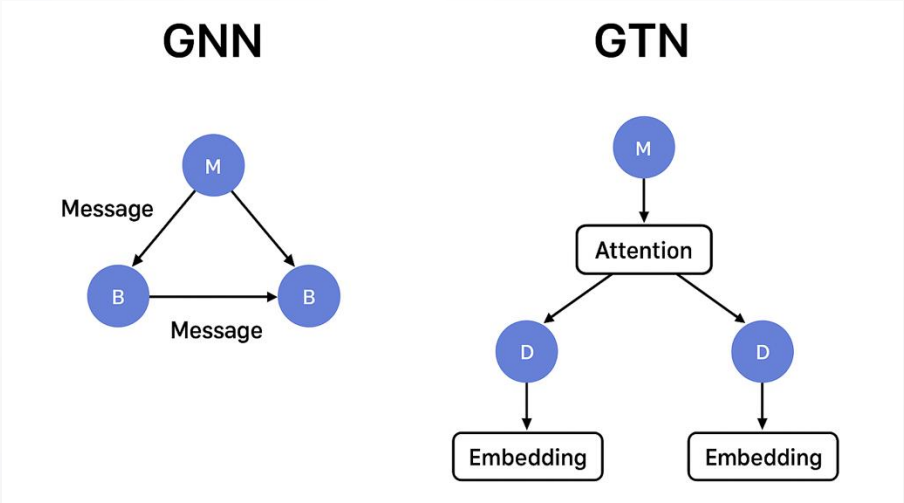
- **Modéliser des graphes hétérogènes ou dynamiques (**Les GTN peuvent facilement s'adapter à des contextes plus complexes) :

  - Graphes multi-types (nœuds et arêtes de nature différente),
  - Graphes évolutifs dans le temps,
  - Graphes implicites.

👉 **Inspirés du MECANISME des Transformers en NLP,** GTN s'inspire des **Transformers** dans le traitement du langage, en appliquant des idées similaires (**attention, position encoding**...) aux données en structure de graphe.

Motivation des Graph Transformers : de GNN à GTN

- Les Graph Neural Networks (GNN) traditionnels ont montré leur efficacité pour exploiter les structures de graphe. Cependant, leurs **limitations** freinent leur performance sur des graphes complexes.
- Contrairement aux GNN classiques qui n'agrègent que les voisins locaux, les GTN apprennent **dynamiquement quelles connexions sont importantes**, même entre des nœuds non directement liés.



GNN vs GTN

Aspect	**GNN classique**	**GTN (Graph Transformer Network)**
Agrégation	Moyenne / somme / max	Attention pondérée sur les voisins
Relations complexes	Limitée	Capturées efficacement
Invariance	Oui (par permutation)	Oui (avec attention)
Expressivité	Moyenne	Plus élevée grâce à l'attention